APIs Web

REST vs. GraphQL vs. SOAP

Programmation comparée — 2026

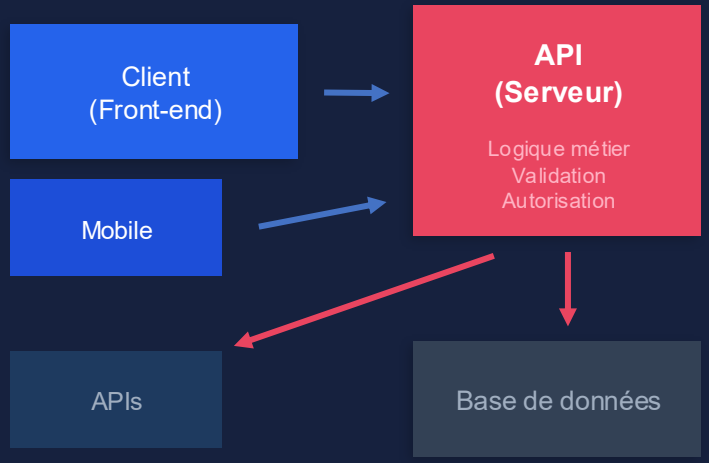
Qu'est-ce qu'une API ?

Application Programming Interface — une interface pour échanger de la donnée entre programmes.

À quoi ça sert ?

1. Interface pour un client pour récupérer et manipuler de la donnée
2. Séparation logique métier / affichage
3. Plusieurs clients différents consomment un même service
4. Contrat : quelles opérations ? quels formats ?

Architecture type



Trois approches, trois époques

Le même problème — exposer des données via le réseau — résolu de trois façons.

1998

SOAP

`Simple Object Access Protocol`

Protocole XML avec contrat formel (WSDL). Intégration entre systèmes hétérogènes en entreprise (Java ↔ .NET ↔ mainframes).

2000

REST

`REpresentational State Transfer`

Style architectural basé sur HTTP. Ressources avec URIs, verbes HTTP sémantiques, réponses cacheables. Thèse de Roy Fielding.

2012

GraphQL

`Graph Query Language`

Langage de requête. Créé par Facebook pour alimenter le News Feed mobile en un seul appel.

SOAP

Simple Object Access Protocol

SOAP — Le concept

Un protocole d'échange de messages XML, indépendant du transport et du langage.

Tout passe par une enveloppe XML

Chaque message est une enveloppe standardisée : Header (sécurité, routing) + Body (opération + paramètres). Le format est identique quel que soit le transport (HTTP, SMTP, JMS).

Contrat formel : le WSDL

Le service est décrit dans un fichier WSDL : opérations disponibles, types de données (XML Schema), protocole de transport. Les clients sont générés automatiquement à partir du contrat.

Sécurité au niveau du message

WS-Security : signature et chiffrement dans le message lui-même (pas juste TLS). Le message reste protégé même à travers des intermédiaires. Standard dans la banque et la santé.

SOAP — L'enveloppe XML

Enveloppe SOAP

Header (optionnel)

WS-Security, routing, transactions

Body

Opération + paramètres typés

Fault (erreurs structurées)

Exemple : GetBook(1)

```
<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap="...soap/envelope">
  <soap:Header>
    <wsse:Security>
      <!-- token, signature -->
    </wsse:Security>
  </soap:Header>
  <soap:Body>
    <GetBook xmlns="...">
      <id>1</id>
    </GetBook>
  </soap:Body>
</soap:Envelope>
```

SOAP — Le WSDL

Web Services Description Language — le contrat machine-readable du service.

Types

Structures de données (XML Schema).
Ex: Book { id, title, year }

Messages

Entrées/sorties de chaque opération.
GetBookRequest(id: int)

Port Types

Interface abstraite : les opérations dispo.
≈ interface Java

Bindings & Service

Protocole concret (SOAP/HTTP) + encodage.
Lie l'abstrait au transport.

Contract-first

Le contrat (WSDL) existe avant le code. Un outil génère automatiquement les classes client dans n'importe quel langage. Le développeur appelle des méthodes typées. Deux organisations intègrent leurs systèmes juste en échangeant un WSDL.

SOAP — Le WSDL

Types

```
<types>
  <xsd:complexType name="Author">
    <xsd:sequence>
     <xsd:element name="Id"   type="xsd:int"/>
     <xsd:element name="Name" type="xsd:string"/>
     <xsd:element name="Bio"  type="xsd:string" />
     <xsd:element name="Nationality" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</types>
```

Messages

```
<message name="GetAuthorRequest">
      <part name="AuthorId" type="xsd:int"/>
</message>
<message name="GetAuthorResponse">
      <part name="Author"   type="tns:Author"/>
</message>
```

SOAP — Le WSDL

Port

```
<portType name="BookCatalogPortType">
   <operation name="GetAuthor">
      <input message="tns:GetAuthorRequest"/>
      <output message="tns:GetAuthorResponse"/>
   </operation>
</portType>
```

Bindings & service

```
<binding name="BookCatalogBinding" type="tns:BookCatalogPortType">
   <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"
   />
   <operation name="GetAuthor">
      <soap:operation soapAction="GetAuthor"/>
   </operation>
</binding>
<service name="BookCatalogService">
   <port name="BookCatalogPort" binding="tns:BookCatalogBinding">
      <soap:address location="http://localhost:3000/soap"/>
   </port>
</service>
```

SOAP — Bilan

Quand l'utiliser

- Banque, santé, défense — WS-Security est un standard
- Transactions distribuées entre services
- Intégration B2B formelle (échange de WSDL)
- Écosystème entreprise (Java EE, .NET, SAP)

Limites

- Verbosité XML — enveloppe > donnée utile
- Complexité de la stack WS-*
- Un seul endpoint POST → pas de cache HTTP
- Mal adapté au web moderne et au mobile

Début 2000, SOAP est omniprésent. Mais le Web évolue, et un style plus simple émerge...

REST

REpresentational State Transfer

REST — Les principes

Un ensemble de principes d'architecture pour des APIs web qui s'intègrent naturellement à HTTP.

Découpe par ressources

Chaque entité (users, books, posts) a sa propre URL

S'appuyer sur les normes HTTP

Verbes (GET, POST, PUT, DELETE) et codes de retour (2XX, 4XX, 5XX) ont une sémantique standard. Pas besoin de réinventer.

Stateless

Pas de mémoire des anciennes requêtes. Chaque requête contient tout son contexte (token, paramètres). Permet le scaling horizontal.

REST — Ressources, verbes, codes

Nomenclature des URLs

```
# Collection
GET  /api/v1/authors
GET  /api/v1/books

# Ressource unique
GET  /api/v1/authors/stephen-king
GET  /api/v1/books/42

# Sous-ressources
GET  /api/v1/books/42/reviews
```

Verbes HTTP = CRUD

GET	Restitution
POST	Création
PUT/PATCH	Modification
DELETE	Suppression

Codes HTTP

2XX — **Succès**
200 OK · 201 Created · 204 No Content

4XX — **Erreur client**
400 Bad Request · 401 Unauthorized · 404 Not Found

5XX — **Erreur serveur**
500 Internal Error · 503 Service Unavailable

REST - HATEOAS

HATEOAS — le niveau ultime

```
GET /books/42

{
  "id": 42,
  "title": "Foundation",
  "_links": {
    "self":    "/books/42",
    "author":  "/authors/1",
    "reviews": "/books/42/reviews"
  }
}
```

Les réponses contiennent des liens vers les actions possibles.
Très peu utilisé en pratique

3	HATEOAS
2	Verbes HTTP
1	Ressources
0	RPC sur HTTP

Richardson Maturity Model

REST - Limites

Le problème : over/under-fetching

```
// Pour 1 page "fiche livre" :

GET /books/42
  → reçoit isbn, pages... (inutile)
GET /authors/1
  → 2e requête pour l'auteur
GET /books/42/reviews
  → 3e requête

= 3 requêtes, données en trop
```

Under-fetching : N requêtes pour 1 vue
Over-fetching : chaque réponse a des champs inutiles

C'est ce problème qui a poussé Facebook à créer GraphQL en 2012.

GraphQL

Graph Query Language

GraphQL - La problématique

Le problème résolu

Facebook 2012 : le News Feed charge des utilisateurs, posts, commentaires, likes, photos

Avec GraphQL : 1 requête, 1 réponse. Le serveur agrège les ressources et renvoie une réponse structurée.

Le client mobile ne gère plus la complexité.

Le News Feed de Facebook en 2012

GraphQL - Le concept

Un langage de requête pour la restitution des données. Le client envoie une requête qui définit exactement les données retournées.

Logique différente de REST : on ne découpe pas par ressources, on définit un modèle de données et des resolvers qui savent aller chercher chaque champ.

Comment ça marche

1. Définition d'un modèle de données typé (schéma)

1. Pour chaque type, un Resolver définit comment récupérer la donnée : SQL, API externe, cache…

1. Un seul endpoint (POST /graphql), le client précise dans sa requête le format et le type de donnée, le serveur la résout champ par champ

1. Introspection : le client peut interroger le schéma

GraphQL — Schéma et resolvers

Le schéma (SDL)

```
type Book {
  id: ID!
  title: String!
  year: Int!
  author: Author!
  reviews: [Review!]!
}

type Query {
  book(id: ID!): Book
  books(limit: Int): [Book!]!
}
```

1 requête HTTP → exactement les données demandées

La requête client

```
query {
  book(id: "42") {
    title
    author { name }
    reviews { rating }
  }
}
```

Le resolver (côté serveur)

```
// Chaque champ a un resolver
Book: {
  author: (book) =>
    db.get("SELECT * FROM authors
      WHERE id = ?", book.author_id),
  reviews: (book) =>
    db.all("SELECT * FROM reviews
      WHERE book_id = ?", book.id)
}
```

GraphQL — Le problème N+1

Chaque resolver est indépendant — chacun fait sa propre requête SQL.

Sans DataLoader (naïf)

```
books(limit:5) { author { name } }

→ SELECT * FROM books LIMIT 5
→ SELECT * FROM authors WHERE id=1
→ SELECT * FROM authors WHERE id=2
→ SELECT * FROM authors WHERE id=3
→ SELECT * FROM authors WHERE id=4
→ SELECT * FROM authors WHERE id=5

= 1 + N requêtes SQL
```

Avec DataLoader (batch)

```
const loader = new DataLoader(
  async (ids) => {
    const rows = db.all(
      `SELECT * FROM authors
       WHERE id IN (${ids})`
    );
    return ids.map(id =>
      rows.find(a => a.id === id)
    );
  }
);

= 2 requêtes SQL seulement
```

GraphQL - Les Inconvénients

- Pas de cache HTTP natif (tout POST)
- Attaques par query complexity
- Pas de support natif du streaming
- Plus difficile à monitorer et logger que REST (pas d'URLs standardisées)
- Upload de fichiers hors spec
- Trop complexe pour des cas simpels

Comparaison

	SOAP	REST	GraphQL
Nature	Protocole	Style architectural	Langage de requête
Format	XML	JSON (ou autre)	JSON
Contrat	WSDL (formel)	OpenAPI (optionnel)	SDL (intrinsèque)
Endpoints	1 (POST)	N (par ressource)	1 (POST /graphql)
Cache HTTP	Difficile	Natif (GET)	Impossible*
Req / vue	1 par opération	N (under-fetching)	1 (client choisit)
Piège	Verbosité XML	Over-fetching	N+1 SQL

** Sauf avec persisted queries ou GET queries*

Démo

Même catalogue de livres avec 3 approches.

1

Récupérer un livre

Comparer XML SOAP
vs JSON REST
vs JSON GraphQL

2

Données liées

REST : 3 requêtes
GraphQL : 1 requête
pour les mêmes données

3

N + 1

Logs SQL : naïf (6)
vs DataLoader (2)
en temps réel

4

Bilan

Tout côte à côte :
req HTTP, req SQL,
taille payloads

S1 — GetBook(id=1)

Trois APIs, même donnée : SOAP renvoie tout, REST overfetch, GraphQL renvoie exactement ce qu'on demande.

SOAP + REST

```
> POST /soap
  3 SQL | 4 128 octets
  (retourne auteur + reviews complets)

> GET /rest/books/1
  1 SQL | 546 octets
  (champs fixes + _links HATEOAS)
```

GraphQL

```
# minimal : seulement certains champs
{
 book(id: "1") { id title year genre }
}
  1 SQL | 130 octets

# complet : auteur + reviews
{
 book(id: "1") {
    title author { name }
    reviews { reviewer rating }
  }
}
  3 SQL | 442 octets
```

S2 — HATEOAS

REST suit ses propres liens : 3 requêtes pour reconstituer ce que GraphQL retourne en une seule.

REST — 3 requêtes

```
> GET /rest/books/1
  1 SQL | 546 octets

> GET /rest/authors/1
  1 SQL | 242 octets

> GET /rest/books/1/reviews
  1 SQL | 657 octets

TOTAL : 3 HTTP, 3 SQL, 1 445 octets
```

GraphQL — 1 requête

```
{
  book(id: "1")
  {
    title year
    author
    {
       name nationality
    }
    reviews
    {
      reviewer rating
    }
  }
}
  3 SQL | 442 octets
```

S3 — Problème N+1

Sans optimisation, GraphQL requête la base une fois par livre tandis qu'avec le DataLoader tout est regroupé dans une seule requête batch.

Requête

```
{
  books(limit 5)
  {
    id
    title
    author
    {
       name
    }
  }
}
```

Sans DataLoader

```
> POST /graphql
  SQL: SELECT * FROM books LIMIT 5
  SQL: SELECT * FROM authors WHERE id=1
  SQL: SELECT * FROM authors WHERE id=2
  SQL: SELECT * FROM authors WHERE id=2
  SQL: SELECT * FROM authors WHERE id=2
  SQL: SELECT * FROM authors WHERE id=1
  6 SQL | 665 octets
```

Avec DataLoader

```
> POST /graphql
  (X-Use-Dataloader: true)
  SQL: SELECT * FROM books LIMIT 5
  SQL: SELECT * FROM authors
       WHERE id IN (1,2)
  2 SQL | 665 octets
```

S4 — Comparaison finale

Objectif : lister 5 livres avec leur auteur (nom + nationalité)

- GraphQL + DataLoader y répond en 1 requête et 2 SQL
- REST en nécessite 3 HTTP et 4 SQL
- SOAP retourne un XML verbeux deux fois plus lourd.

REST + SOAP

```
REST (liste + 2 auteurs) :
  3 req   4 SQL   2 048 octets
  1+N HTTP, assemblage côté client

SOAP GetBooks :
  1 req   2 SQL   1 580 octets
  XML verbeux, schéma rigide
```

GraphQL + DataLoader

```
{
  books(limit: 5) {
    id title genre year
    author {
       name nationality
    }
  }
}
  1 req   2 SQL   665 octets
  1 HTTP, payload minimal,
  exactement ce qui est demandé
```

Quand utiliser quoi ?

Il n'y a pas de gagnant universel.

SOAP

- WS-Security (banques, santé)
- Transactions distribuées
- Intégration B2B (WSDL)
- Écosystème Java EE, .NET

REST

- CRUD, ressources bien définies
- Cache HTTP (CDN)
- APIs publiques
- Simplicité et adoption large

GraphQL

- Clients variés (mobile, web, IoT)
- Données interconnectées
- Itération rapide frontend
- Projection fine des données

Merci !

REST · GraphQL · SOAP

Questions ?

Sources

1. Fielding (2000). Architectural Styles and the Design of Network-Based Software Architectures. UC Irvine.

2. W3C. SOAP 1.2 Specification. https://www.w3.org/TR/soap12/

3. GraphQL Foundation. GraphQL Specification. https://spec.graphql.org/

4. Fowler (2010). Richardson Maturity Model. https://martinfowler.com/articles/richardsonMaturityModel.html

5. Byron (2015). GraphQL: A data query language. https://engineering.fb.com/core-infra/graphql-a-data-query-language/

6. W3C. WSDL 2.0 Specification. https://www.w3.org/TR/wsdl20/

7. Two-Bit History (2020). Fielding's Misappropriated REST Dissertation. twobithistory.org

8. GraphQL Foundation. DataLoader. https://github.com/graphql/dataloader